

Compte-rendu SAÉ 302 : Développer des applications communicantes

Présenté par : Catarino ANTONIO, Ilyas HADADI, Tarek BOUDIF, Paul-Emile NGUYEN-TAN-HON

Avec l'aide de Mr SPIES



Année 2024-2025

IUT Nord Franche-Comté

BUT Réseaux & Télécommunication



SAE Développer des applications communicantes

Introduction

Dans le cadre du projet SAE 302, nous avons développé une application client-serveur qui simule un petit réseau social. Ce projet avait pour objectif de nous apprendre à utiliser les communications entre plusieurs utilisateurs grâce au protocole UDP, en utilisant le langage Java et NetBeans.

L'application permet à un groupe d'utilisateurs d'échanger des messages en temps réel et de les envoyer comme dans une boîte aux lettres. Nous avons donc décidé de créer notre application Dark Mango une application de messagerie facile à déployer et accessible à tous.

Dark Mango est une application de messagerie client-serveur développée en Java avec NetBeans. Elle simule un petit réseau social où les utilisateurs peuvent s'inscrire, se connecter, envoyer et lire des messages, et gérer des amis. Elle repose sur le protocole **UDP**, choisi pour sa simplicité et sa rapidité.

L'application est composée de deux programmes :

- ClientUDP.java : application graphique Swing utilisée par les utilisateurs pour interagir avec le serveur.
- ServeurUDP.java : serveur en console qui gère les demandes client (authentification, messagerie, amis)

Le travail a été réparti sur plusieurs jours avec des tâches bien définies : planification, développement, tests, et préparation de la présentation finale. Chaque membre de l'équipe a contribué selon un rôle précis pour garantir le bon déroulement du projet.

Pour ce rapport nous avons travaillé en groupe de 4 Catarino ANTONIO, Ilyas HADADI, Tarek BOUDIF, Paul-Emile NGUYEN-TAN-HON.

Présentation de la gestion de projet

Le projet a été réalisé par 4 étudiants (Catarino Antonio, Ilyas Haddadi, Tarek Boudif, Paul-Emile Nguyen-Tan-Hon) dans le cadre de la SAE 302. Nous avons suivi une démarche en 4 étapes :

1. Planification : choix des rôles, architecture et tâches.
2. Développement : répartition claire entre interface, logique serveur et gestion réseau.
3. Tests : vérification de chaque fonctionnalité (connexion, envoi, demande).
4. Présentation : rapport, tutoriel, préparation orale.

Chaque membre s'est concentré sur une partie, tout en coordonnant le projet en binômes.

Fonctionnalités demandées

L'application est conçue pour gérer jusqu'à 4 utilisateurs simultanément, chacun identifié par une adresse email et un mot de passe stockés en clair. Les messages échangés suivent une structure spécifique sous forme de chaîne de caractères, comprenant une commande ou une action (par exemple, **INVITATION** ou **MESSAGE**), une origine (l'émetteur), un destinataire, un sujet, et un corps.

Les messages sont stockés et pour chaque utilisateur, seuls les 10 derniers messages reçus sont conservés. Les messages plus anciens sont automatiquement supprimés pour limiter l'utilisation de la mémoire. Les messages stockés

Les interactions entre les utilisateurs incluent :

1. L'ajout d'amis via une commande de type **demande ami**, avec une notification de confirmation envoyée par le serveur lorsque l'invitation est acceptée.
2. L'envoi de messages, qui peuvent être destinés à un utilisateur précis ou à tous les amis de l'émetteur (grâce à la commande **EVERYONE**).

Ces fonctionnalités, basées sur une architecture client-serveur utilisant les sockets Java, assurent un système d'échange interactif tout en respectant les contraintes techniques liées à la mémoire et à la simplicité du protocole.

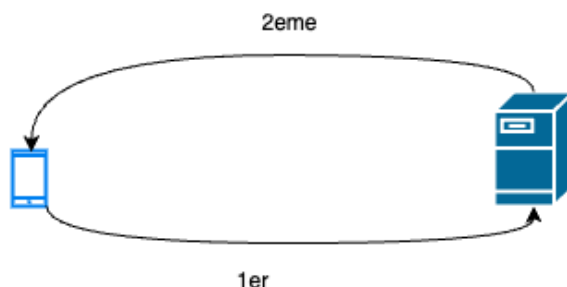
Pourquoi UDP ?

- Simplicité de mise en place (DatagramSocket)
- Pas besoin de connexions persistantes
- Moins de surcharge qu'en TCP
- Facilement déployable
- Charge légère

On peut voir ci-dessous un schéma représentant les principales fonctionnalités de notre messagerie et les interactions entre le Client et le Serveur :

1. Les requêtes émises par le Client vers le Serveur (Client → Serveur).
2. Les réponses ou actions envoyées par le Serveur au Client (Serveur → Client).

Ce schéma a été réalisé à l'aide de l'outil Lucidchart, qui permet de connaître les flux de communication entre les différentes parties du système.



COMPTE

A :

- 1 - create,user,mdp
- 2a - create-accept
- 2b - credentials-requis
- 2c - login-deja-existant
- 2d - max-user-atteint

B :

- 1 - connect,user,mdp
- 2a - connect-accept
- 2b - login-refuse
- 2c - mdp-refuse
- 2d - credentials-requis

MESSAGE :

C :

- 1 - lire,user
- 2a - message
- 2b - user-not-found
- 2c - no-message

D :

- 1 - send-message,user,ami,contenu
- 2a - message-envoye
- 2b - no-friend
- 2c - champ-ami-requis
- 2d - champ-contenu-requis
- 2e - destinataire-introuvable
- 2f - pas-ami-avec-destinatiare
- 2g - max-message

AMI :

E :

- 1 - add-ami,user,ami
- 2a - demande-send
- 2b - champ-ami-vide
- 2c - ami-not-found
- 2d - user-same-ami
- 2e - deja-ami

F :

- 1 - liste-ami,user
- 2a - listing-ami
- 2b - no-ami

G :

- 1 - demande-ami,user
- 2a - listing-demande
- 2b - no-request

H :

- 1 - demande-attente,user,ami,accept/refuse
- 2a - accepted
- 2b - refused
- 2c - invalid-request
- 2d - max-user-from-you
- 2e - max-user-from-friend
- 2f - no-request-from-friend

Fonctionnalités principales et secondaires :

L'implémentation actuelle repose sur une interface graphique côté client, qui offre plusieurs fonctionnalités essentielles :

- Création de compte avec des identifiants uniques (email et mot de passe).
- Vérification de la syntaxe du mail.
- Connexion et déconnexion.
- Ajout d'amis en envoyant une demande spécifique.
- Consultation de la liste d'amis
- Envoi de messages, à des amis précis ou à tous les amis simultanément via la commande **EVERYONE**.
- Lecture des messages reçus (10 derniers maximum)

Lors de la connexion, le serveur valide les informations transmises par le client. Si les identifiants (email et mot de passe) sont corrects, le serveur confirme la réussite de la requête au client. En cas d'échec, le serveur retourne une réponse détaillant la nature de l'erreur :

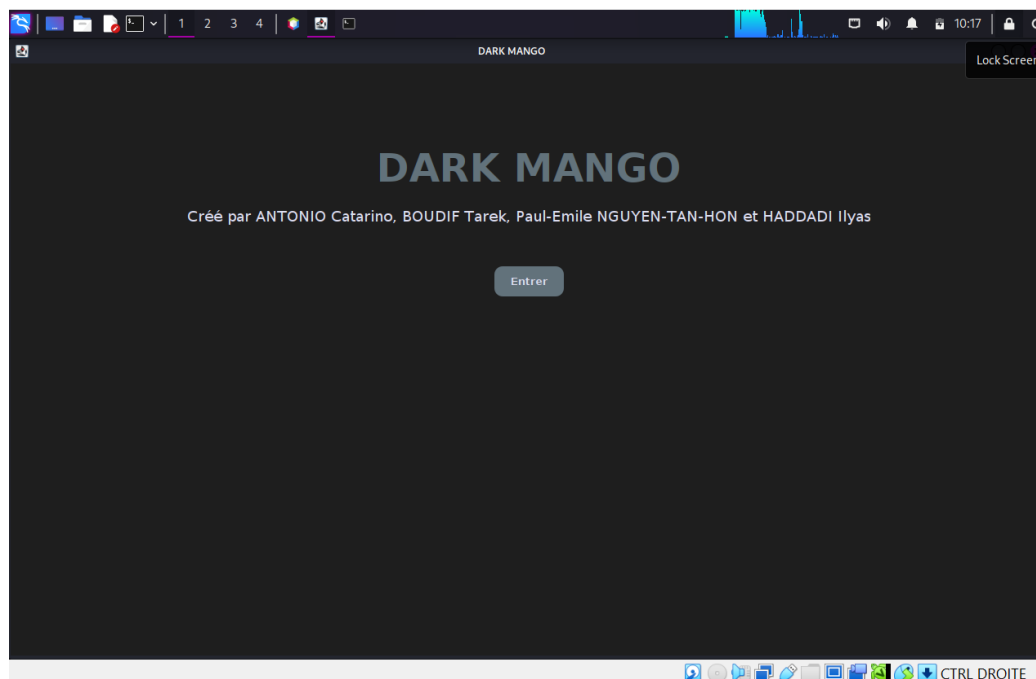
- Si le mot de passe est incorrect, une notification spécifique est envoyée.

- Dans d'autres cas d'erreur, le contenu complet du paquet reçu est affiché pour permettre une analyse approfondie.

On peut voir ci-dessous le serveur démarré sur le port 3333 ce qui va nous permettre de communiquer avec les différents clients :

```
Output - serveurUDP (run)
run:
Serveur UDP démarré sur le port 3333
```

Interface du serveur au lancement



Interface Client au Lancement de l'Application

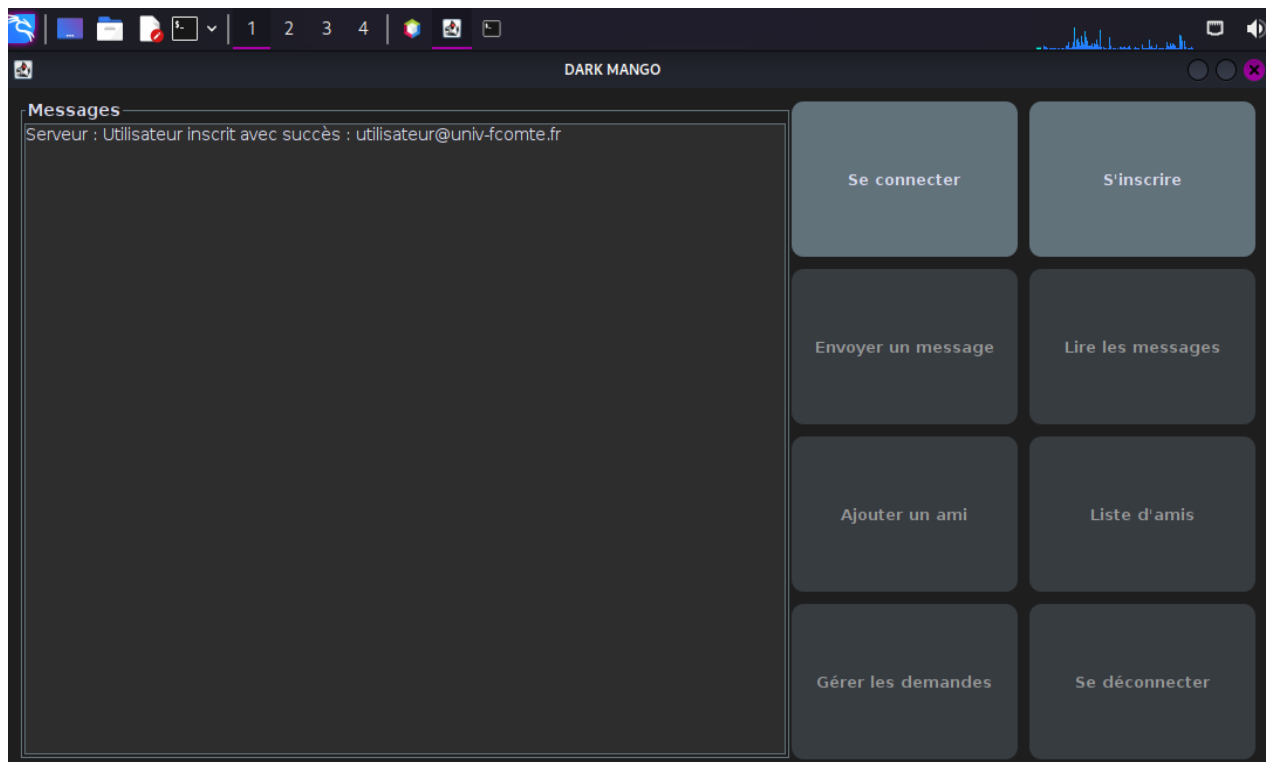
Tutoriel :

Au démarrage de l'application, l'interface client présente plusieurs fonctionnalités, mais certaines d'entre elles, comme **Lire un message**, **Envoyer un message**, ou d'autres options similaires, sont volontairement désactivées. Cette décision a été prise afin de prévenir certains bugs potentiels et d'assurer une meilleure stabilité du système. Pour accéder à ces fonctionnalités, l'utilisateur doit d'abord :

1. **S'inscrire** : créer un compte avec une adresse email valide et un mot de passe.
2. **Se connecter** : s'authentifier avec les identifiants créés.

Une fois connecté, les boutons et les fonctionnalités sont débloqués, permettant ainsi à l'utilisateur d'envoyer des messages et d'utiliser pleinement l'application. Cette approche garantit une gestion plus sécurisée et structurée des interactions utilisateur.

Une fois connecté l'utilisateur peut ajouter des amis, avec le bouton ajouter un ami ou accepter/refuser des demandes avec les boutons gérer des demandes. Une fois ami, vous êtes donc autorisé à envoyer des messages entre ami. Vous pouvez consulter vos messages avec le bouton « Lire les messages » et consulter votre liste d'ami avec le bouton « Liste d'ami ».



Interface après lancement

Conclusion

Ce projet de SAE 302 nous a permis de mettre en pratique nos compétences en développement d'applications communicantes, en particulier grâce à l'utilisation du langage Java et du protocole UDP. En créant une application client-serveur simulant un petit réseau social, nous avons appris à concevoir des échanges structurés entre utilisateurs tout en respectant des contraintes techniques précises comme la gestion de la mémoire ou la sécurisation des accès.

Ce travail collaboratif nous a non seulement permis de renforcer nos compétences techniques, mais également de développer notre capacité à travailler en groupe, à planifier et à adapter nos choix face aux imprévus.

En résumé, cette SAE a été une expérience formatrice, tant sur le plan technique qu'humain, et nous a permis d'aboutir à une application fonctionnelle répondant aux objectifs initiaux du projet.